| | |
|---|---|
| Started on | Friday, 24 April 2026, 10:31 AM |
| State | Finished |
| Completed on | Friday, 24 April 2026, 10:36 AM |
| Time taken | 5 mins 58 secs |
| Name | JEBA BLESSYBHA A 2025-BM5 |

Question 1

Correct

Marked out of 1.00

Flag question

An argument is an expression which is passed to a function by its caller in order for the function to perform its task. It is an expression in the comma-separated list bound by the parentheses in a function call expression.

A function may be called by the portion of the program with some arguments and these arguments are known as actual arguments (or) original arguments.

Actual arguments are local to the particular function. These variables are placed in the function declaration and function call. These arguments are defined in the calling function.

The parameters are variables defined in the function to receive the arguments.

Formal parameters are those parameters which are present in the function definition.

Formal parameters are available only with in the specified function. Formal parameters belong to the called function.

Formal parameters are also the local variables to the function. So, the formal parameters are occupied memory when the function execution starts and they are destroyed when the function execution completed.

Let us consider the below example:

```
include <stdio.h>
nt add(int, int);
nt main()

       int a = 10, b = 20;
       printf("Sum of two numbers = %d\n", add(a, b)); // variables a, b are called actual argur
       return 0;


nt add(int x, int y)

      // variables x, y are called formal parameters
      return(x + y);
```

In the above code whenever the function call add(a, b) is made, the execution control is transferred to the function definition of add().

The values of actual arguments a and b are copied in to the formal arguments x and y respectively.

The formal parameters x and y are available only with in the function definition of add(). After completion of execution of add(), the control is transferred back to the main().

See & retype the below code which will demonstrate about formal and actual arguments.

```
#include <stdio.h>

int sum(int);

int main()
{
   int number;
   scanf("%d", &number);
   printf("Sum of %d natural numbers = %d\n", number, sum(number));
   return 0;
}

int sum(int value)
{
   int i, total = 0;
   for (i = 1; i <= value; i++)
   {
      total = total + i;
   }
   return(total);
}
```

For example:

| Input | Result |
|---|---|
| 5 | Sum of 5 natural numbers = 15 |

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
int sum(int);
int main()
{
    int number;
    scanf("%d",&number);
    printf("Sum of %d natural numbers = %d\n" ,number,sum(number));
    return 0;
}
    int sum(int value)
{
    int i,total = 0;
    for(i = 1;i<=value;i++)
    {
        total = total + i;
    }
    return (total);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 5 | Sum of 5 natural numbers = 15 | Sum of 5 natural numbers = 15 | ✔ |

Passed all tests! ✔

| | |
|---|---|
| Started on | Friday, 24 April 2026, 10:38 AM |
| State | Finished |
| Completed on | Friday, 24 April 2026, 10:52 AM |
| Time taken | 13 mins 36 secs |
| Name | JEBA BLESSYBHA A 2025-BM5 |

Question 1

Correct

Marked out of 1.00

Flag question

Local variables are declared and used inside a function (or) in a block of statements.

Local variables are created at the time of function call and destroyed when the function execution is completed.

Local variables are accessible only with in the particular function where those variables are declared.

Global variables are declared outside of all the function blocks and these variables can be used in all functions.

Global variables are created at the time of program beginning and reside until the end of the entire program.

Global variables are accessible in the entire program.

If a local and global variable have the same name, then local variable has the highest precedence to access with in the function.

Let us consider an example:

```
#include <stdio.h>
void change();
int x = 20; // Global Variable x
int main()
{
        int x = 10; // Local Variable x
        change();
        printf("%d", x); // The value 10 is printed
        return 0;
}
void change()
{
        printf("%d", x); // The value 20 is printed
}
```

In the above code the global and local variables have the same variable name x, but they are different.

In main() function the local variable x is only accessed, so it prints the value 10.

In change() function the variable x is not declared locally so it access global variable x, so it prints 20.

See & retype the below code which will demonstrate about local and global variables.

```
#include <stdio.h>

int x = 15;

void change1(int x)
{
   printf("In change1() function x = %d\n", x);
}

void change2()
{
   printf("In change2() function x = %d\n", x);
}

int main()
{
   int x = 10;
   printf("In main() function x = %d\n", x);
   change1(x);
   change2();
   printf("In main() function x = %d\n", x);
   return 0;
}
```

For example:

| Result |
|---|
| In main() function x = 10<br>In change1() function x = 10<br>In change2() function x = 15<br>In main() function x = 10 |

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
int x = 15;
void change1(int x)
{
    printf("In change1() function x = %d\n",x);
}
void change2()
{
    printf("In change2() function x = %d\n",x);
}
int main()
{
    int x = 10;
    printf("In main() function x = %d\n",x);
    change1(x);
    change2();
    printf("In main() function x = %d\n",x);
    return 0;
}
```

| | Expected | Got | |
|---|---|---|---|
| ✔ | In main() function x = 10<br>In change1() function x = 10<br>In change2() function x = 15<br>In main() function x = 10 | In main() function x = 10<br>In change1() function x = 10<br>In change2() function x = 15<br>In main() function x = 10 | ✔ |

Passed all tests! ✔

Question 2

Correct

Marked out of 1.00

Flag question

Global variables are declared outside of any function.

A global variable is visible to any every function and can be used by any piece of code.

Unlike local variable, global variables retain their values between function calls and throughout the program execution.

Let us consider an example:

```
#include <stdio.h>
int a = 20; // Global declaration
void test();
int main()
{
        printf("In main() function a = %d\n", a); // Prints 20
        test();
        a = a + 15; // Uses global variable
        printf("In main() function a = %d\n", a); // Prints 55
        return 0;
}
void test()
{
        a = a + 20; // Uses global variable
        printf("In test() function a = %d\n", a); // Prints 40
}
```

In the above code the global variable a is declared outside of all the functions. So, the variable a can be accessed in every function.

Operating System calls the main() function at the time of execution. the variable a has no local declaration, so it access the global variable a.

In test() function also there is no local declaration of variable a, the variable a gets access from the global.

The global variables are destroyed only after completion of execution of entire program.

See & retype the below code which will demonstrate about global variables.

```
#include <stdio.h>

int a = 20;

void test();

int main()
{
   printf("In main() function a = %d\n", a);
   test();
   a = a + 15;
   printf("In main() function a = %d\n", a);
   return 0;
}

void test()
{
   a = a + 20;
   printf("In test() function a = %d\n", a);
}
```

For example:

| Result |
| --- |
| In main() function a = 20<br>In test() function a = 40<br>In main() function a = 55 |

Answer: (penalty regime: 0 %)

```
1  #include<stdio.h>
2  int a = 20;
3  void test();
4  int main()
5  {
6      printf("In main() function a = %d\n",a);
7      test ();
8      a = a + 15;
9      printf("In main() function a = %d\n",a);
10     return 0;
11 }
12 void test()
13 {
14     a = a+20;
15     printf("In test() function a = %d\n",a);
16 }
```

| | Expected | Got | |
| --- | --- | --- | --- |
| ✔ | In main() function a = 20<br>In test() function a = 40<br>In main() function a = 55 | In main() function a = 20<br>In test() function a = 40<br>In main() function a = 55 | ✔ |

Question 3

Correct

Marked out of 1.00

Flag question

A local variable is declared inside a function.

A local variable is visible only inside their function, only statements inside function can access that local variable.

Local variables are declared when the function execution started and local variables gets destroyed when control exits from function.

Let us consider an example:

```
#include <stdio.h>
void test();
int main()
{
        int a = 22, b = 44;
        test();
        printf("Values in main() function a = %d and b = %d\n", a, b);
        return 0;
}

void test()
{
        int a = 50, b = 80;
        printf("Values in test() function a = %d and b = %d\n", a, b);
}
```

In the above code we have 2 functions main() and test(), in these functions local variables are declared with same variable names a and b but they are different.

Operating System calls the main() function at the time of execution. the local variables with in the main() are created when the main() starts execution.

when a call is made to test() function, first the control is transferred from main() to test(), next the local variables with in the test() are created and they are available only with in the test() function.

After completion of execution of test() function, the local variables are destroyed and the control is transferred back to the main() function.

See & retype the below code which will demonstrate about local variables.

```
#include <stdio.h>

void test();

int main()
{
   int a = 9, b = 99;
   test();
   printf("Values in main() function a = %d and b = %d\n", a, b);
   return 0;
}

void test()
{
   int a = 5, b = 55;
   printf("Values in test() function a = %d and b = %d\n", a, b);
}
```

For example:

| Result |
| --- |
| Values in test() function a = 5 and b = 55<br>Values in main() function a = 9 and b = 99 |

Answer: (penalty regime: 0 %)

```
1  #include<stdio.h>
2  void test();
3  int main()
4  {
5      int a = 9,b = 99;
6      test();
7      printf("Values in main() function a = %d and b = %d\n",a,b);
8      return 0;
9  }
10 void test()
11 {
12     int a = 5,b = 55;
13     printf("Values in test() function a = %d and b = %d\n",a,b);
14 }
```

| | Expected | Got | |
| --- | --- | --- | --- |
| ✔ | Values in test() function a = 5 and b = 55<br>Values in main() function a = 9 and b = 99 | Values in test() function a = 5 and b = 55<br>Values in main() function a = 9 and b = 99 | ✔ |

Passed all tests! ✔

| | |
|---|---|
| Started on | Friday, 24 April 2026, 10:53 AM |
| State | Finished |
| Completed on | Friday, 24 April 2026, 11:08 AM |
| Time taken | 15 mins 42 secs |
| Name | JEBA BLESSYBHA A 2025-BM5 |

Question 1

Correct

Marked out of 1.00

Flag question

Fill the missing code to understand the concept of a function with arguments and without return value.

Note: Take pi value as 3.14

The below code is to find the area of circle using functions.

For example:

| Input | Result |
|---|---|
| 11.23 | Area of circle = 395.994476 |

Answer: (penalty regime: 0 %)

Reset answer

```c
#include <stdio.h>

void area_circle(float);

int main()
{
    float radius;
    scanf("%f", &radius);
    area_circle(radius);
    return 0;
}

void area_circle(float r)
{
    //Correct the code
    float area;
    area = 3.14*r*r;
    // Write the code to calculate the area of circle
    printf("Area of circle = %f\n", area);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 11.23 | Area of circle = 395.994476 | Area of circle = 395.994476 | ✔ |

Passed all tests! ✔

Question 2

Correct

Marked out of 1.00

Flag question

Fill in the missing code in the below code to understand about function with arguments and with return value.

The below code is to find the factorial of a given number using functions.

For example:

| Input | Result |
|---|---|
| 3 | Factorial of a given number 3 = 6 |

Answer: (penalty regime: 0 %)

Reset answer

```c
#include <stdio.h>

int factorial(int);

int main()
{
    int number;
    scanf("%d", &number);
    printf("Factorial of a given number %d = %d\n", number, factorial(number)
    return 0;
}

int factorial(int n)
{
    int i, fact = 1;
    for (i = 1;i <= n;i++)
    {
        fact = fact*i;// Write code to calculate the factorial of a given num
    }
    return fact;
    // Write the return statement
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3 | Factorial of a given number 3 = 6 | Factorial of a given number 3 = 6 | ✔ |

Passed all tests! ✔

Question 3

Correct

Marked out of 1.00

Flag question

When a function definition has arguments, it receives data from the calling function.

After taking some desired action, only one value will be returned from called function to calling function through the return statement.

If a function returns a value, the function call may appear in any expression and the returned value used as an operand in the evaluation of the expression.

Let us consider an example of a function with arguments and with return value:

```
#include <stdio.h>
int largest(int, int, int);
int main()
{
        int a, b, c;
        printf("Enter three numbers : ");
        scanf("%d%d%d" , &a, &b, &c);
        printf(" Largest of the given three numbers = %d\n", largest(a, b, c));
        return 0;
}
int largest(int x, int y, int z)
{
        if ((x > y) && (x > z))
        {
                return x;
        }
        else if (y > z)
        {
                return y;
        }
        else
        {
                return z;
        }
}
```

In the above sample code the function int largest(int, int, int); specifies that the function receives three values and returns a value to the calling function.

Fill in the missing code in the below program to find the largest of three numbers using largest() function.

For example:

| Input | Result |
|---|---|
| 99 49 29 | Largest of the given three numbers = 99 |
| 45 67 35 | Largest of the given three numbers = 67 |

Answer: (penalty regime: 0 %)

Reset answer

```
1  #include <stdio.h>
2
3  int largest(int, int, int);
4
5  int main()
6  {
7      int a, b, c;
8      scanf("%d%d%d", &a, &b, &c);
9      printf("Largest of the given three numbers = %d\n", largest(a,b,c)); // C
10     return 0;
11 }
12
13 int largest(int x,int y,int z)
14 {
15     // Correct the code
16     if (x>y && x>z)
17     {
18         // Correct the code
19         return x ; // Correct the code
20     }
21     else if (y>z)
22     {
```

| | Input | Expected | Got |
|---|---|---|---|
| ✔ | 99 49 29 | Largest of the given three numbers = 99 | Largest of the given three numbers = 9 |
| ✔ | 45 67 35 | Largest of the given three numbers = 67 | Largest of the given three numbers = 6 |

Passed all tests! ✔

Question 4

Correct

Marked out of 1.00

Flag question

When a function has no arguments, it does not receive any data from the calling function.

When a function return a value, the calling function receives data from the called function.

Let us consider an example of a function without arguments and with return value:

```
#include <stdio.h>
int sum(void);
int main()
{
        printf("\nSum of two given values = %d\n", sum());
        return 0;
}
int sum() {
        int a, b, total;
        printf("Enter two numbers : ");
        scanf("%d%d", &a, &b);
        total = a + b;
        return total;
}
```

In the above sample code the function int sum(void); specifies that the function does not receive any arguments but return a value to the calling function.

Fill in the missing code in the below program to find sum of two integers.

For example:

| Input | Result |
|---|---|
| 9 5 | Sum of two given values = 14 |
| 45 78 | Sum of two given values = 123 |

Answer: (penalty regime: 0 %)

Reset answer

```
#include <stdio.h>

int sum(void);

int main()
{
    printf("Sum of two given values = %d\n", sum());
    return 0;
}

int sum()
{
    int a,b,total;
    scanf("%d %d",&a,&b);
    total = a+b;
    return total;
    // Fill in the missing code
    // Read two integers
    // Find sum
    // Retun sum
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 9 5 | Sum of two given values = 14 | Sum of two given values = 14 | ✔ |
| ✔ | 45 78 | Sum of two given values = 123 | Sum of two given values = 123 | ✔ |

Passed all tests! ✔

Question 5

Correct

Marked out of 1.00

Flag question

Write a C program to demonstrate functions without arguments and with return value.

The below code is used to check whether the given number is a prime number or not.

Write the function prime().

Sample Input and Output:

```
5
The given number is a prime number
```

For example:

| Input | Result |
|---|---|
| 5 | The given number is a prime number |
| 27 | The given number is not a prime number |
| 121 | The given number is not a prime number |
| 1 | The given number is not a prime number |

Answer: (penalty regime: 0 %)

Reset answer

```c
#include <stdio.h>

int prime();

int main()
{
    if (prime() == 0)
    {
        printf("The given number is a prime number\n");
    }
    else
    {
        printf("The given number is not a prime number\n");
    }
    return 0;
}
int prime(){
    int n,i,count = 0;
    scanf("%d",&n);
    if(n <= 1)
    return 1;

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 5 | The given number is a prime number | The given number is a prime number | ✔ |
| ✔ | 27 | The given number is not a prime number | The given number is not a prime number | ✔ |
| ✔ | 121 | The given number is not a prime number | The given number is not a prime number | ✔ |
| ✔ | 1 | The given number is not a prime number | The given number is not a prime number | ✔ |

Passed all tests! ✔

Question 6

Correct

Marked out of 1.00

Flag question

Write a C program to demonstrate functions without arguments and without return value.

Write the functions print() and hello().

The output is:

```
...***...
Hello! REC
...***...
```

For example:

| Result |
| --- |
| ...***...<br>Hello! REC<br>...***... |

Answer: (penalty regime: 0 %)

Reset answer

```c
#include <stdio.h>
void print()
{
    printf("...***...\n");
}
void hello()
{
    printf("Hello! REC\n");
}

// Write the functions

int main()
{
    print();
    hello();
    print();
    return 0;
}
```

| | Expected | Got | |
| --- | --- | --- | --- |
| ✔ | ...***...<br>Hello! REC<br>...***... | ...***...<br>Hello! REC<br>...***... | ✔ |

Passed all tests! ✔

Question 7

Correct

Marked out of 1.00

Flag question

When a function definition has arguments, it receives data from the calling function.

The actual arguments in the function call must correspond to the formal parameters in the function definition, i.e. the number of actual arguments must be the same as the number of formal parameters, and each actual argument must be of the same data type as its corresponding formal parameter.

The formal parameters must be valid variable names in the function definition and the actual arguments may be variable names, expressions or constants in the function call.

The variables used in actual arguments must be assigned values before the function call is made. When a function call is made, copies of the values of actual arguments are passed to the called function.

What occurs inside the function will have no effect on the variables used in the actual argument list. There may be several different calls to the same function from various places with a program.

Let us consider an example of a function with arguments and without return value:

```
#include <stdio.h>
void largest(int, int);
int main()
{
        int a, b;
        printf("Enter two numbers : ");
        scanf("%d%d" , &a, &b);
        largest(a, b);
        return 0;
}
void largest(int x, int y)
{
        if (x > y)
        {
                printf("Largest element = %d\n", x);
        }
        else
        {
                printf("Largest element = %d\n", y);
        }
}
```

In the above sample code the function void largest(int, int); specifies that the function receives two integer arguments from the calling function and does not return any value to the called function.

When the function call largest(a, b) is made in the main() function, the values of actual arguments a and b are copied in to the formal parameters x and y.

After completion of execution of largest(int x, int y) function, it does not return any value to the main() function. Simply the control is transferred to the main() function.

Fill in the missing code in the below program to find the largest of two numbers using largest() function.

For example:

| Input | Result |
|---|---|
| 27 18 | Largest element = 27 |
| 13 17 | Largest element = 17 |

Answer: (penalty regime: 0 %)

Reset answer

```
#include <stdio.h>

void largest(int x, int y);

int main()
{
    int a, b;
    scanf("%d%d", &a, &b);
    largest(a,b); // Correct the code
    return 0;
}

void largest(int x,int y)
{
    // Correct the code
    if (x>y)
    {
        // Correct the code
        printf("Largest element = %d\n", x);
    }
    else
    {
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 27 18 | Largest element = 27 | Largest element = 27 | ✔ |
| ✔ | 13 17 | Largest element = 17 | Largest element = 17 | ✔ |

Passed all tests! ✔

Question 8

Correct

Marked out of 1.00

Flag question

All the C functions can be called either with arguments or without arguments in a C program. These functions may or may not return values to the calling function.

Depending on the arguments and return values functions are classified into 4 categories.

1. Function without arguments and without return value
2. Function with arguments and without return value
3. Function without arguments and with return value
4. Function with arguments and with return value

When a function has no arguments, it does not receive any data from the calling function.

Similarly, when a function does not return a value, the calling function does not receive any data from the called function.

In effect, there is no data transfer between the calling function and the called function in the category function without arguments and without return value.

Let us consider an example of a function without arguments and without return value:

```
#include <stdio.h>
void india_capital(void);
int main()
{
        india_capital();
        return 0;
}
void india_capital()
{
        printf("New Delhi is the capital of India\n");
}
```

In the above sample code the function void india_capital(void); specifies that the function does not receive any arguments and does not return any value to the main() function.

Identify the below errors and correct them.

For example:

| Result |
| --- |
| New Delhi is the capital of India |

Answer: (penalty regime: 0 %)

Reset answer

```
1  #include <stdio.h>
2
3  void india_capital();
4
5  int main()
6  {
7      india_capital();
8      return 0;
9  }
10
11 void india_capital()
12 {
13     printf("New Delhi is the capital of India\n");
14 }
```

| | Expected | Got | |
| --- | --- | --- | --- |
| ✔ | New Delhi is the capital of India | New Delhi is the capital of India | ✔ |

Passed all tests! ✔